LASR-MCP: Learned Adaptive Schema Resolution with Dynamic In-Tree Recovery for Enterprise Tool-Calling Agents

Anand S. Suchak
Independent Researcher
Email: anand02suchak@gmail.com

Abstract—As enterprise computing transitions toward autonomous agentic workflows, the Model Context Protocol (MCP) has emerged as the standard open interface for tool calling and context integration. However, deploying complex enterprise schemas (such as multi-constraint purchase order systems) introduces severe operational penalties: monolithic full-schema exposure inflates context token consumption, while naive schema pruning triggers catastrophic argument hallucinations due to omitted domain constraints.

To resolve this tension, we present Learned Adaptive Schema Resolution (LASR-MCP), a three-tier architecture that decouples tool definitions into: (1) an Invariant Foundation (C_I^*) ensuring a structural safety floor, (2) a Proactive Intent Predictor pre-allocating task-conditioned constraint bundles, and (3) a Dynamic In-Tree Recomposition Operator that reactively expands schemas upon validation errors via pass-level multi-error extraction.

We evaluate LASR-MCP across a synthetic enterprise-style inventory dispatching benchmark ($N = 32$ tasks across four operational archetypes) under gemini-3.5-flash-lite: First, on an unseen template family, our semantic intent predictor achieves an F_1 score of 0.863 versus 0.250 for frequency baselines ($\Delta F_1 = +0.612$, $p < 0.001$). Second, within the benchmark evaluation, reactive in-tree recovery provides empirical evidence consistent with a causal contribution over static adaptive exposure, yielding a +21.9 percentage point structural reliability improvement (78.1% $\rightarrow$ 100.0%, 95% CI: $[+9.4, +37.5]$ pp, $p < 0.01$), successfully recovering all 27 initially failed trajectories within a three-attempt budget. Crucially, reactive recovery terminates wasteful retry cascades early, achieving a 20.2% to 22.1% reduction in total workload API tokens compared to monolithic full-schema exposure ($96,629 \rightarrow 77,140$ tokens at $k = 1$, and $75,248$ tokens at $k = 2$), identifying $k = 2$ as the best observed operating point. All benchmark data, traces, and code are publicly archived with a SHA-256 cryptographic manifest for deterministic trace replay and build verification.

Index Terms—Model Context Protocol (MCP), Autonomous Agents, Adaptive Schema Resolution, JSON Schema, Enterprise Tool-Calling, In-Tree Recovery, Token Efficiency.

I. Introduction

The deployment of Large Language Models (LLMs) in mission-critical enterprise workflows has shifted from conversational assistants toward autonomous, closed-loop execution agents [1], [2]. In enterprise operations—such as multi-warehouse replenishment, inventory reconciliation, and procurement logistics—agents interact with transactional backends via standardized tool-calling protocols. The Model Context Protocol (MCP) [3] (formalized in specification 2026-07-28) formalizes this client-server interaction, providing structured JSON-RPC interfaces for tool discovery and parameter submission. Our experiments target MCP tool-schema exposure and tool-call argument generation; protocol transport, session, and discovery semantics are outside the evaluation scope.

A. The Dilemma of Enterprise Tool Schemas

Despite MCP's architectural standardization, exposing enterprise-grade tools to LLMs presents a challenging systems dilemma:

1) The Monolithic Full-Schema Tax: Enterprise tools enforce intricate data contracts governed by JSON Schema Draft 2020-12 [4] (e.g., supplier regex patterns, item-level quantity bounds, warehouse routing enums, and budget restrictions). Transmitting complete monolithic schemas (often exceeding 500 tokens per tool) on every turn inflates prompt token overhead, drives up financial API costs, and degrades inference throughput [5].
2) The Brittleness of Naive Truncation: Conversely, statically stripping schemas down to basic primitive types eliminates vital domain rules. In our benchmarks, when task-conditioned rules (such as warehouse routing enums) are omitted, models hallucinate non-existent identifiers, resulting in catastrophic validation failures that may remain unresolved through unassisted retries.

B. Our Approach: Three-Tier Adaptive Resolution

To overcome this trade-off, we propose Learned Adaptive Schema Resolution (LASR-MCP). Rather than treating schema resolution as static compression or isolated error string dumping, LASR-MCP introduces a tripartite schema hierarchy:

$$C_{\text{active}}^{(t)} = C_I^* \cup \hat{C}_T(x) \cup C_R^{(t)} \tag{1}$$

combining:

- Tier 1 (Invariant Foundation Floor, C_I^*): A core subset of constraints (339 tokens for the evaluated purchase-order schema) that establishes the non-negotiable structural foundation required for syntactically valid submissions.

- Tier 2 (Proactive Intent Prediction, $\hat{C}_T(x)$): A lightweight semantic classifier that predicts task-relevant constraint bundles from user intent prior to execution.
- Tier 3 (Dynamic In-Tree Recomposition, $C_R^{(t)}$): A reactive operator that extracts the full set of concurrent schema violations from validator feedback and splices the missing constraint nodes directly into the native JSON Schema hierarchy on Attempt 2.

C. Hypotheses & Empirical Findings

We investigate four central hypotheses across a synthetic enterprise-style supply-chain tool benchmark:

- H_1 (Semantic Predictability): Task intent contains sufficient signal to predict constraint requirements beyond frequency priors on unseen template families. → Strongly Supported: LASR achieves $F_1 = 0.863$ versus 0.250 for the frequency prior ($\Delta F_1 = +0.612$, $p < 0.001$).
- H_2 (Invariant Safety Floor): A minimal invariant foundation is necessary to prevent structural collapse. → Supported within the Evaluated Benchmark: $C_I^* = 339$ tokens is the empirically selected invariant foundation meeting the pre-specified 70% structural-validity floor; further pruning drops structural validity from 75% to 50%.
- H_3 (Reactive In-Tree Recovery): Pass-level dynamic in-tree recovery provides empirical evidence consistent with a causal contribution over static adaptive exposure. → Strongly Supported: Structural success increases from 78.1% to 100.0% (ΔSuccess = +21.9 pp, 95% CI: $[+9.4, +37.5]$ pp, exact one-sided McNemar/binomial test $p = 0.0078125$, $p < 0.01$), recovering all 27 initially failed trajectories in the benchmark.
- H_4 (Workload Token Efficiency): Adaptive resolution reduces net API token consumption across enterprise workloads. → Supported for Workload-Level Tokens: Reactive LASR cuts total API token consumption by 20.2% ($k = 1$) and 22.1% ($k = 2$) compared to monolithic full-schema control ($96,629 \rightarrow 75,248$ tokens).

D. Contributions

1) Formalization of Three-Tier Schema Resolution: We formalize the interaction between invariant structural baselines, proactive intent prediction, and pass-level in-tree recomposition.
2) Controlled Systems Benchmark: We execute a six-arm controlled systems evaluation across 32 tasks spanning four operational archetypes on the primary frozen benchmark, providing empirical evidence consistent with a causal contribution of reactive recovery.
3) Open Reproducibility Package: We release the benchmark code, traces, and frozen dataset with an automated SHA-256 cryptographic integrity manifest.

II. The LASR-MCP Architecture

To resolve the trade-off between bloated context overhead and brittle schema truncation, we introduce Learned Adaptive Schema Resolution (LASR-MCP). Rather than treating tool schemas as static monolithic declarations or appending unanchored constraint slices, LASR-MCP formulates the active tool schema at execution attempt t as a three-tier union:

$$C_{\text{active}}^{(t)} = C_I^* \cup \hat{C}_T(x) \cup C_R^{(t)} \tag{2}$$

where C_I^* represents the schema-wide Invariant Foundation, $\hat{C}_T(x)$ is the Proactive Task-Conditioned Constraint Bundle predicted from user intent x, and $C_R^{(t)}$ is the accumulated set of Dynamically Recovered In-Tree Constraints derived from validation feedback up to round t.

A. Tier 1: Invariant Foundation Floor (C_I^*)

Empirical inspection of complex JSON Schema specifications (e.g., enterprise purchase order dispatching) reveals that tool execution cannot succeed on intent-specific properties alone. Certain structural constraints are invariants required for all syntactically well-formed submissions regardless of user goal (e.g., supplier identification regex format, item array container specifications, and SKU patterns).

Let $\mathcal{C}_{\text{full}}$ denote the complete schema constraint set (552 tokens). We define the Invariant Foundation C_I^* as the minimal constraint subset among tested candidates satisfying the pre-specified baseline structural-validity floor without user-intent conditioning:

$$C_I^* = \arg\min_{C \in \mathcal{C}_{\text{cand}}} |C| \quad \text{s.t.} \quad \mathcal{V}_{\text{struct}}(C) \geq \tau_{\text{floor}} \tag{3}$$

where $\mathcal{C}_{\text{cand}}$ is the set of evaluated candidate invariant subsets and $\tau_{\text{floor}} = 0.70$ specifies the baseline validity floor, chosen a priori as the minimum operational viability threshold to avoid retry cascade failure. As demonstrated in Section IV, $C_I^* = 339$ tokens is the empirically selected invariant foundation meeting this threshold among tested tiers for the evaluated purchase-order schema; compressing below this boundary triggers structural collapse in nested item definitions.

B. Tier 2: Proactive Intent-Conditioned Predictor ($\hat{C}_T(x)$)

Given a user query $x \in \mathcal{X}$, the agent's goal is to predict the minimal subset of task-conditioned constraints $\hat{C}_T(x) \subseteq \mathcal{C}_{\text{task}}$ necessary for successful execution. We formulate this as a multi-label classification problem over the task-conditioned label space $\mathcal{L} = \{c_1, c_2, \ldots, c_m\}$ (e.g., specific warehouse routing enums, order priority restrictions, and budget bounds).

We employ a frozen linear semantic predictor:

$$\hat{y}_i = \sigma\left(\mathbf{w}_i^T \phi(x) + b_i\right), \quad i \in \{1, \ldots, m\} \tag{4}$$

where $\phi(x)$ denotes normalized term-salience features extracted from the query. The decision threshold θ was calibrated strictly on the 48 training tasks and frozen prior to evaluation. To construct the proactive bundle of size up to k, the highest-ranked constraints exceeding θ (up to k) are selected:

$$\hat{C}_T(x) = \text{Top-}k\left(\{c_i \in \mathcal{L} \mid \hat{y}_i \geq \theta\}\right) \tag{5}$$

C. Tier 3: Dynamic In-Tree Recomposition ($C_R^{(t)}$)

When an unpredicted constraint is omitted from $C_I^* \cup \hat{C}_T(x)$, the LLM may submit arguments y_t that fail JSON Schema validation. A simple agent implementation may return raw validator error strings into conversation history or retry without modifying the exposed schema.

LASR-MCP introduces in-tree reactive recomposition. Reactive recovery operates as pass-level multi-error constraint expansion rather than single-error sequential repair. Our validator is configured to return all observed violations $\text{Errors}(y_t) = \{e_1, e_2, \ldots\}$ in a single validation pass, rather than short-circuiting on the first failure. Analogous to an enterprise validator reporting all diagnostic issues across a payload, the reactive operator extracts all concurrent violations simultaneously:

$$C_R^{(t+1)} = C_R^{(t)} \cup \bigcup_{e \in \text{Errors}(y_t)} \left(\text{Fragment}(e) \cap \mathcal{S}_{\text{specs}} \setminus C_{\text{active}}^{(t)}\right) \tag{6}$$

where $\text{Fragment}(e)$ deterministically maps the RFC 6901 JSON Pointer error path [6] and validator-reported JSON Schema keyword (e.g., path='/warehouse_code', keyword='enum') to the canonical constraint node $\Delta_e \in \mathcal{S}_{\text{specs}}$.

The updated schema specification S_{t+1} is then structurally spliced into the native JSON Schema hierarchy:

$$S_{t+1} = \mathcal{R}\left(S_I, C_{\text{active}}^{(t+1)}\right) \tag{7}$$

where $\mathcal{R}$ is the in-tree recomposition function that inserts properties, pattern regexes, and enums directly into their respective object trees without creating duplicate definition nodes or schema syntax errors.

D. Incremental Token Accounting Metrics

To evaluate token economics, we audit token consumption at every turn:

- Initial Schema Overhead (S_0): Tokens required to serialize $S_{\text{active}}^{(1)}$.
- Incremental Expansion (ΔS_t): Additional schema tokens introduced at attempt $t \geq 2$: $\Delta S_t = \text{Tok}(S_t) - \text{Tok}(S_{t-1})$.
- Tokens-to-Success (TTS): Cumulative tokens across all attempts until successful completion, defined strictly over successful trajectories ($T_{\text{succ}} \leq 3$):

$$TTS = \sum_{t=1}^{T_{\text{succ}}} \left(P_t + \text{Tok}(S_t) + C_t\right) \tag{8}$$

  where P_t denotes non-schema prompt tokens (system instructions, non-schema tool metadata, user prompt, and conversation history), $\text{Tok}(S_t)$ denotes serialized schema tokens, and C_t denotes output completion tokens (preventing schema double-counting).
- Total Workload Tokens: Sum of all tokens consumed across all 32 tasks and all attempts up to the three-attempt budget, including failed retry loops.
- Structural Recovery Rate: Fraction of initially structurally invalid trajectories that subsequently satisfy JSON Schema validation within the attempt budget.

For workload-token reporting, we use the provider-reported totalTokenCount for each API request. The component fields promptTokenCount, candidatesTokenCount, thoughtsTokenCount, and toolUsePromptTokenCount are retained for audit and are not independently summed into the workload total; serialized schema tokens are separately audited for architectural decomposition.

III. Experimental Setup

A. Task Corpus & Stratified Template Splits

We evaluate LASR-MCP on a synthetic enterprise-style supply-chain tool execution benchmark (create_purchase_order) governing multi-item warehouse replenishment. The benchmark corpus comprises 80 pre-specified task instances partitioned strictly across syntactic template families:

- Training Tasks ($N = 48$): Derived from template families family_a and family_b, used to train the frozen semantic predictor.
- Evaluation Tasks ($N = 32$): Drawn exclusively from family_c with distinct lexical and syntactic structures (serving as unseen held-out test tasks for H_1 intent prediction, and development/validation tasks for H_2–H_4 systems evaluation).

The 32 evaluation tasks span four diverse operational archetypes ($N = 8$ tasks each):

1) C1: Routine Regional Replenishment: High-frequency, standard priority orders governed by basic warehouse routing.
2) C2: Expedited Emergency Orders: Critical priority dispatches with accelerated freight logistics and rigid budget limits.
3) C3: Specialized Bulk Procurement: Large-quantity orders where queries omit explicit warehouse locations, testing model behavior under unstated default enums.

4) C4: Multi-Item Depot Rebalancing: Complex multi-line consignments enforcing nested maxItems and item-level SKU validation.

B. Controlled Experimental System Arms

To compare the contributions of the schema-resolution components in Equation (2), we evaluate six controlled system arms under identical prompt conditions:

- Full Schema Control (full): Static exposure of the monolithic 552-token schema upfront.
- Invariant Baseline (ci_k0): Invariant Foundation alone ($C_I^* = 339$ tokens) without task-conditioned prediction or reactive recovery.
- Proactive Static Control (ci_k1_static): Invariant Foundation + Top-1 LASR predicted constraint without reactive recovery.
- Proactive-Reactive LASR (ci_k1_reactive): Invariant Foundation + Top-1 LASR predicted constraint with dynamic in-tree recovery.
- Proactive-Reactive LASR (ci_k2_reactive): Invariant Foundation + Top-2 LASR predicted constraints with dynamic in-tree recovery.
- Model-Conditional Reference (oracle_reactive): Invariant Foundation + reference constraint set $O(x) = C_I^* \cup C_T^*(x)$ with reactive recovery. $O(x)$ was constructed from task specifications prior to execution and validated against the schema validator.

C. Dual Evaluation Criteria

Submissions are evaluated on two orthogonal metrics:

1) Structural Schema Validity (%): Conformance to JSON Schema Draft 2020-12 verified by an independent automated validator.
2) Semantic Task Correctness (%): Exact domain logic validation verifying that extracted line items, quantities, priority codes, and supplier IDs satisfy user intent. Evaluation is strictly hierarchical: semantic correctness requires structural validity as a precondition, bounding semantic correctness by structural success.

D. Execution Environment

All experiments run using model ID gemini-3.5-flash-lite hosted on Google Cloud Generative Language API via direct REST HTTP endpoints. Generation used temperature 0.0 with fixed decoding seeds where supported; each recorded API response stores the provider-reported modelVersion alongside usage metadata.

E. Evaluation Hygiene & Benchmark Scope

To maintain strict methodological hygiene, we explicitly delineate the empirical role of each benchmark split:

- Held-Out Family Generalization for H_1: The multi-label linear predictor was trained on 48 tasks (family_a, family_b) and evaluated on 32 tasks (family_c) drawn from an unseen syntactic template family, evaluating held-out generalization across template families. The predictor weights were formally frozen prior to all subsequent experiments.
- Primary Frozen Benchmark for H_2–H_4: The invariant foundation $C_I^* = 339$ tokens was calibrated on the family_c task distribution. Consequently, the 32-task controlled systems benchmark evaluates system behavior within this primary frozen benchmark corpus rather than serving as an untouched, blind end-to-end generalization test.

F. Data Provenance & Trace Replay Verification

All benchmark code, traces, and frozen datasets are archived at https://github.com/AnandSuchak/mcp-inventory-bench. While live API re-executions may exhibit variance from hosted model updates, deterministic reproduction of our experimental results is guaranteed via recorded execution traces verified by an automated SHA-256 cryptographic manifest:

SHA-256:
78c2c88ed5128e0f296fa29e0c258589b516f3adb14b0b8da491fa73bc3ffc11
Frozen Benchmark Commit:
9807340abc38170e0090a102547ee4309c01d78b

Automated regression test suites ensure complete data immutability.

IV. Empirical Results & Discussion

We structure our empirical analysis around four core research hypotheses evaluated across the primary 32-task frozen benchmark corpus.

A. H_1: Semantic Predictability over Frequency Baselines

We first evaluate whether intent-driven multi-label classification can reliably anticipate task-conditioned constraints on unseen template families compared to an empirical frequency prior. The frequency prior ranks constraint labels by empirical prevalence across the 48 training tasks and selects the top-k most frequent constraints. On the unseen template family evaluation ($N = 32$ from family_c), LASR at $k = 2$ achieves an F_1 score of 0.863 and a 68.8% exact bundle match, compared to $F_1 = 0.250$ and 25.0% exact match for the Frequency Prior baseline. We resampled the 32 evaluation tasks with replacement for $B = 2,000$ paired bootstrap replicates, computing percentile bootstrap confidence intervals:

$$\Delta F_1 = +0.612 \quad [95\% \text{ CI: } +0.431, +0.775], \quad p < 0.001 \tag{9}$$

This strongly supports H_1, demonstrating that task text contains predictive signal for constraint pre-allocation across template families.

B. H_2: Invariant Foundation Requirement on the Evaluated Benchmark

To identify the minimal invariant floor C_I^* meeting the pre-specified 70% structural-validity threshold ($\tau_{\text{floor}} =$

TABLE I
Full-Spectrum Systems Evaluation across 32 Development/Validation Tasks ($N = 32$, Gemini 3.5 Flash-Lite)

System Arm	Reactive?	Init S_0	Exp. ΔS	1st-Pass (%)	Struct. (%)	Sem. (%)	Mean TTS	Total Tokens
Full Schema Control	No	552.0	0.0	31.2%	93.8%	90.6%	2913.2	96,629
C_I^* Baseline ($k = 0$)	No	339.0	0.0	0.0%	68.8%	65.6%	3182.9	102,407
LASR $k = 1$ (Static)	No	361.1	0.0	15.6%	78.1%	71.9%	2410.6	83,754
LASR $k = 1$ (Reactive)	Yes	361.1	45.4	15.6%	100.0%	93.8%	2410.6	77,140
LASR $k = 2$ (Reactive)	Yes	366.8	43.8	18.8%	100.0%	93.8%	2351.5	75,248
Oracle $O(x)$ (Reactive)*	Yes	359.0	48.3	15.6%	96.9%	93.8%	2325.3	75,649

*Note: Oracle $O(x)$ is a model-conditional reference baseline. Mean TTS is averaged over successful trajectories (N_{succ}); Total Tokens accounts for all attempts across all 32 tasks.

0.70), we followed a formal sequential design: pre-specify threshold → candidate calibration → freeze C_I^* → final 32-task systems evaluation. During candidate calibration on an 8-task calibration probe, we benchmarked three candidate compression tiers prior to task conditioning:

- Tier 1 (Standard C_I^*, 339 tokens): 75.0% structural validity (6/8 tasks).
- Tier 2 (Compact C_I, 308 tokens): 62.5% structural validity (5/8 tasks).
- Tier 3 (Minimal C_I, 284 tokens): 50.0% structural validity (4/8 tasks).

Tier 1 ($C_I^* = 339$ tokens) was the smallest tested candidate meeting the 70% floor and was frozen for all subsequent experiments. In the subsequent full 32-task systems benchmark (Table I), this frozen foundation alone without task prediction (ci_k0) yielded 68.8% structural validity (22/32 tasks), confirming that without task-conditioned augmentation or reactive recovery, structural validity hovers near the 70% baseline floor. Compressing below 339 tokens triggers progressive degradation due to missing definitions for nested items. We emphasize that 339 tokens represents the smallest tested candidate meeting this threshold, not an unconstrained universal constant.

C. H_3: Evidence Consistent with a Causal Contribution of Reactive Recovery

TABLE II
Paired Confidence Intervals and Hypothesis Tests ($N = 32$ Development/Validation Tasks)

Pairwise Contrast	Metric	Point Est.	95% Bootstrap CI	Significance Test (p)
LASR Reactive vs Static	Δ Structural Succ.	+21.9 pp	[9.4, 37.5] pp	Exact one-sided McNemar/binomial ($p = 0.0078125$)
	Δ Semantic Corr.	+21.9 pp	[9.4, 37.5] pp	Exact one-sided McNemar/binomial ($p = 0.0078125$)
LASR Reactive vs Full	Δ Structural Succ.	+6.2 pp	[0.0, 15.6] pp	Exact one-sided McNemar/binomial ($p = 0.250$)
	Δ Semantic Corr.	+3.1 pp	[-6.2, 12.5] pp	Exact one-sided McNemar/binomial ($p = 0.500$)
Workload Token Lift ($k = 1$)	Total API Tokens	-20.2%	-19,489 tokens	Paired Empirical Workload
Workload Token Lift ($k = 2$)	Total API Tokens	-22.1%	-21,381 tokens	Paired Empirical Workload

The central systems question is whether reactive in-tree recovery provides empirical evidence consistent with a causal contribution over static adaptive exposure. In Table I, we contrast ci_k1_static directly with ci_k1_reactive:

Structural Reliability: Final structural success increases from 78.1% to 100.0% (ΔSuccess = **+21.9** pp, 95% CI: [+9.4, +37.5] pp, exact one-sided McNemar/binomial test $p = 0.0078125$).

Semantic Correctness: Task correctness increases from 71.9% to 93.8% (ΔCorrectness = **+21.9** pp, 95% CI: [+9.4, +37.5] pp, exact one-sided McNemar/binomial test $p = 0.0078125$).

Structural Recovery Rate: All 27 initially failed trajectories in the benchmark were successfully recovered to satisfy structural validation within the three-attempt budget ($27/27 = 100.0\%$).

As broken down in Table III, the observed lift is concentrated in Cluster 3 (Specialized Bulk Procurement), where static success was only 50.0% (4/8 tasks) due to missing warehouse enums, while reactive recovery achieved 100.0% (8/8 tasks).

TABLE III
Task Archetype Breakdown: Static Vulnerability vs. Reactive Recovery ($N = 8$ per Cluster)

Task Cluster Archetype	Full (%)	Static $k = 1$	Reactive $k = 1$	Observed Lift
C1: Routine Regional	87.5%	100.0%	100.0%	0.0 pp
C2: Expedited Emergency	100.0%	87.5%	100.0%	+12.5 pp
C3: Specialized Bulk	87.5%	50.0%	100.0%	+50.0 pp
C4: Multi-Item Depot	100.0%	75.0%	100.0%	+25.0 pp

D. H_4: Total Workload Token Efficiency

Table I highlights the macro token economics of adaptive resolution across the 32-task workload: Full Schema Control consumed 96,629 total API tokens with a mean TTS of 2,913.2 tokens, while reactive LASR cut consumption to 77,140 tokens at $k = 1$ (-20.2%) and 75,248 tokens at $k = 2$ (-22.1% net token reduction).

Observed Correctness vs. Full Schema: In this benchmark, reactive LASR achieved 93.8% semantic correctness compared to 90.6% for Full Schema (+3.1 pp). Because LLM outputs exhibit inherent stochasticity, this difference reflects an observed empirical result on this benchmark distribution rather than evidence that schema reduction intrinsically enhances underlying semantic comprehension. The core claim rests on reliability preservation and workload token reduction.

Why Reactive LASR Outperforms Static LASR in Total Tokens: In ci_k1_static, failing tasks cycle through 3 wasted attempts without converging, accumulating 83,754 tokens. In ci_k1_reactive, because in-tree schema expansion resolves the root cause immediately on Attempt 2, the

system terminates the retry cascade early, saving 6,614 tokens (-7.9%) relative to static LASR.

E. Emergence of $k = 2$ as the Best Observed Operating Point

While $k = 1$ provides the leanest initial schema ($S_0 = 361.1$ tokens), ci_k2_reactive ($S_0 = 366.8$ tokens) achieved the lowest total token consumption overall (75,248 tokens) while achieving 100% structural validity and 93.8% semantic correctness. By predicting slightly wider constraint bundles ($k = 2$) upfront, the system avoids initial failures on Cluster 2 without adding meaningful schema bloat.

F. Decomposing Token Savings: Schema Reduction vs. Retry Avoidance

A natural systems question is: How much of the total token savings stems from a leaner schema footprint versus avoiding failed retry turns? Auditing granular token accounts across the 32 tasks reveals the exact operational balance:

Initial Schema Savings vs. Expansion Cost: In ci_k2_reactive, total schema serialization consumed 24,972 tokens compared to 34,776 for Full Schema—a net saving of 9,804 schema tokens (-28.2%), even after paying 1,402 tokens of amortized reactive expansion across all attempts (43.8 tokens/task).

Retry-Avoidance Savings: In ci_k1_static, failing tasks cycle through 3 attempts without converging, accumulating 75 total attempts across the workload. In ci_k1_reactive, in-tree schema expansion resolves omitted rules immediately on Attempt 2, reducing workload attempts to 66 (avoiding 9 full failure rounds). This retry avoidance saves 4,059 prompt tokens and 1,294 completion tokens (5,353 tokens), plus 1,261 avoided schema tokens, yielding 6,614 net API tokens saved (-7.9%) over the static arm.

Net Workload Impact: Comparing ci_k2_reactive against Full Schema, 45.9% of the 21,381 saved tokens (9,804 tokens) derives from leaner schema serialization, while 54.1% (11,577 tokens) derives from prompt and completion token savings during multi-turn interactions.

G. Auditable Trajectory Case Study

Figure 1 details the step-by-step recovery trajectory for task BLIND-C3-013. On Attempt 1, exposed to C_I^* + top-1 LASR rule (347 tokens), the model generates unconstrained enum 'WH-EAST-01'. The schema validator flags three concurrent violations (warehouse_code.enum, order_priority.enum, supplier_id.pattern). LASR intercepts these error paths and recomposes the schema tree in-tree (347 → 419 tokens). On subsequent attempts, the agent resolves all constraints, converging to 100% structural and semantic correctness.

Task: BLIND-C3-013 (Specialized Bulk) Arm: ci_k1_reactive

[Attempt 1]
- Exposed Schema: $C_I^* \cup$ {items.maxItems} (347 tokens)
- Model Output: warehouse_code = 'WH-EAST-01'
- Validator: FAIL (3 concurrent violations in pass):

1. 'WH-EAST-01' not in ['WH-US-EAST', …]
2. 'standard' not in ['STANDARD', …]
3. 'SUP-ACME1234-US' does not match regex

- Extracted: {warehouse.enum, priority.enum, supplier.pattern}

[In-Tree Recomposition] Spliced Schema: 347 → 419 tok (+72 tok)
[Attempt 2] Model Output: 'WH-US-EAST' (Enum Resolved)
[Attempt 3] Output: Valid purchase order JSON
- Validator: PASS (Structural: 100%, Semantic: 100%)
- Outcome: RECOVERED (3 attempts, $TTS = 3,626$ tok)

Fig. 1. Trajectory-level evidence of pass-level multi-error extraction and native in-tree schema recomposition on family_c evaluation task BLIND-C3-013.

V. Post-Freeze Cross-Schema Robustness Evaluation

To evaluate cross-schema transfer beyond purchase orders, we conducted a post-freeze robustness study across three new synthetic schemas plus primary replication (four schemas total) under JSON Schema Draft 2020-12 [4]: (1) create_purchase_order (Procurement: 552 full / 339 C_I^* tok; regexes, enums, budget bounds, items); (2) inventory_adjustment (Warehouse: 541 / 299 tok; signed deltas, facilities, reason codes); (3) shipment_creation (Logistics: 646 / 337 tok; carriers, tracking, packages, weights); and (4) customer_refund (Finance: 474 / 292 tok; refund limits, currencies, return classifications).

A. Evaluation Methodology and Scrutiny Controls

For each schema, we constructed 8 pre-specified evaluation tasks ($N = 32$ total tasks), partitioned into two operational regimes: Explicit Tasks (16 tasks, 4 per schema), where intents furnish required enum tokens and parameters; and Ambiguous Tasks (16 tasks, 4 per schema), intentionally constructed as a failure-induction stress test without explicit cues, starting with lean C_I^* to induce initial violations and test closed-loop reactive recomposition.

Methodological controls: (1) all 32 tasks, schemas, and invariant floors were pre-specified prior to model execution or inspection; (2) no task was altered post-hoc; (3) no predictor weight was tuned on new domains; and (4) the primary frozen benchmark ($N = 32$, commit 9807340, SHA-256 78c2c88e) was kept quarantined and untouched.

B. Scope of Transfer: Architecture vs. Predictor

Transfer scope across schemas: 1) Architecture Transfer: Closed-loop reactive recovery—error interception, keyword extraction, and in-tree recomposition—operates domain-agnostically. 2) Schema Foundations: Each schema instantiates its invariant floor ($C_I^* \in \{339, 299, 337, 292\}$ tok) and dictionary $\mathcal{S}_{\text{specs}}$, fixed a priori. 3) Predictor Status: We evaluate recovery transfer without retraining or retuning the primary predictor; explicit tasks test intent-conditioned rules, while ambiguous tasks test zero-shot recovery from C_I^*. This evaluates architecture and

TABLE IV
Cross-Schema Robustness Evaluation across Four Heterogeneous Enterprise-Style Domains ($N = 32$ Tasks total, 8 per Schema)

Enterprise-Style Tool Schema	Domain / Key Constraints	Init S_0	Full S	Token Savings	1st-Pass (Exp/Amb)	Final Struct.	Semantic	Ambiguous Recovery
create_purchase_order	Procurement (Regex, enums, nested items)	339	552	-21.3%	50.0% (4/4, 0/4)	100.0%	87.5%	100.0% (4/4)
inventory_adjustment	Warehouse Stock (Signed deltas, reason codes)	299	541	-22.4%	50.0% (4/4, 0/4)	100.0%	87.5%	100.0% (4/4)
shipment_creation	Freight Logistics (Nested packages, tracking regex)	337	646	-25.2%	50.0% (4/4, 0/4)	100.0%	100.0%	100.0% (4/4)
customer_refund	Financial Accounting (Bounds, currency enums)	292	474	-20.5%	50.0% (4/4, 0/4)	100.0%	100.0%	100.0% (4/4)
Macro Mean / Aggregate	Cross-Domain Summary ($N = 32$)	316.8	553.2	-22.5%	50.0% (16/16, 0/16)	100.0%	93.8%	100.0% (16/16)

mechanism transfer, rather than universal predictor transfer across arbitrary unseen APIs.

C. Empirical Robustness Results

Table IV summarizes system performance across all four enterprise-style domains:

Domain Token Savings: Across the three new synthetic domains, LASR consistently reduces cumulative token consumption: saving 22.4% on inventory_adjustment, 25.2% on shipment_creation, and 20.5% on customer_refund (macro mean: 22.7%), alongside 21.3% on procurement replication (unrounded macro mean: 22.4%; aggregate workload reduction: -22.5%). On multi-turn ambiguous tasks requiring reactive disambiguation, a compact foundation ($C_I \approx 290$–340 tok) plus targeted reactive delta ($\Delta S \approx 40$ tok) consumes substantially fewer tokens than transmitting monolithic schemas across every turn.

Complete Ambiguous Task Recovery (16/16): The 50.0% first-pass rate (16/16 explicit, 0/16 ambiguous) reflects the 50/50 stress-test design. On all 16 ambiguous tasks where first-pass rejections occurred from unconstrained parameters, the multi-error extractor intercepted diagnostics and recomposed missing subtrees into the schema tree, achieving a 100.0% structural recovery rate on Attempt 2 and 100.0% final structural compliance across all 32 tasks (representing observed stress-test performance, not an unconstrained universal guarantee).

High Semantic Fidelity: Evaluated via independent deterministic domain validators checking operational fidelity (routing, quantities, carrier assignment, currency), LASR attained 87.5% to 100.0% semantic correctness (macro aggregate: 93.8%), verifying that compact schema exposure preserves domain execution fidelity.

These findings provide preliminary evidence of cross-schema transfer: LASR's core architectural mechanics transfer effectively across diverse enterprise-style JSON Schema contracts without retuning the recovery algorithm or the primary predictor; schema-specific constraint vocabularies and invariant foundations were instantiated for each schema.

VI. Related Work and Novelty Positioning

We situate LASR-MCP within four emerging paradigms in tool-augmented LLM literature:

1) Tool Retrieval in MCP: Tool catalogs exhaust context windows as registries grow. RAG-MCP [7] and MCP-Zero [8] employ dense retrieval to filter catalogs, while Gorilla [5] and ToolLLM [9] retrieve APIs from large sets. Distinction: These frameworks operate at the inter-tool catalog level (which tools to load), whereas LASR-MCP operates intra-tool, resolving parameter constraints once a tool is chosen.

2) Schema Compression and Context Efficiency: Tool Attention [10] lazily loads parameter docs via intent gating. TSCG [11] compiles verbose schemas into dense interfaces; prompt compression (LLMLingua [12], LLMLingua-2 [13]) and recent evaluations show tool-schema compression enables agentic workflows under constrained context budgets. Distinction: TSCG changes the representation of the schema at the API boundary, whereas LASR retains the native JSON Schema representation for the exposed constraints, adaptively exposing and recomposing constraint nodes within an already selected tool while enforcing an Invariant Foundation Floor (C_I^*).

3) Field-Aware and Schema Routing: SchemaRouter [14] evaluates field-aware routing, noting under-fetching risks, while HyperAgent [15] plans over schema hypergraphs. Distinction: SchemaRouter routes fields across an API graph; LASR resolves validation constraints inside an already selected executable tool and expands those constraints from execution-time validator feedback, achieving complete structural recovery across evaluated cases.

4) Execution Traces and Tool Adaptation: ReAct [1] and Toolformer [2] demonstrate self-correction, and ToolSpec [16] uses speculative decoding. Distinction: A simple trace-based recovery strategy may append raw validator errors to conversation history; LASR-MCP intercepts validator paths for in-tree JSON Schema recomposition, splicing missing constraint subtrees directly into the schema on Attempt 2.

Summary of Novelty: LASR-MCP resolves constraints within an already selected tool schema, combining intent-conditioned proactive prediction with validator-derived in-tree recomposition.

VII. Discussion, External Validity, and Limitations

The empirical findings from our primary frozen benchmark (H_1–H_4) and cross-schema study demonstrate that learned adaptive schema resolution with reactive recovery provides an efficient, reliable foundation for enterprise tool agents. Below, we analyze architectural boundaries, validity threats, and operational trade-offs.

A. Schema Topologies and Structural Expressiveness

Enterprise APIs exhibit diverse topologies; our evaluation incorporated nested objects and arrays (e.g.,

line items in create_purchase_order, packages in shipment_creation). While JSON Pointer traversal handles nested definitions deterministically, polymorphic branches (oneOf, anyOf) introduce pruning risks if discriminators are omitted. In LASR, discriminator fields and structural keys are designated as invariants (C_I^*), ensuring schema tree branching remains unambiguous. Automated DAG analysis for recursive schemas remains future work.

B. Schema Validator Standards vs. Business Logic

A core design philosophy of LASR-MCP is strict adherence to JSON Schema Draft 2020-12 specifications [4]. Our configured validator reports error locations as JSON Pointer paths [6] and validation keywords; LASR maps these diagnostics to schema-specific canonical constraint nodes without bespoke heuristics. Enterprise backends also enforce transactional business rules (e.g., stock levels, credit checks) outside schema syntax; LASR-MCP explicitly scopes itself to syntactic and structural schema conformance, leaving business failures to application compensation.

C. Model-Family Sensitivities and Parameter Scale

Controlled experiments benchmarked Google's gemini-3.5-flash-lite as a representative enterprise model. Pilot ablations observed unconstrained enum generation across evaluated configurations, suggesting the failure mode is not unique to the evaluated primary configuration. For small open-weight models, lean schemas (C_I^*) prevent attention dilution; for frontier models, they mitigate multi-turn token costs. In both regimes, LASR's lean baseline and closed-loop fallback provide operational advantages.

D. Epistemological Separation and Benchmark Provenance

We emphasize strict separation of empirical scopes: 1) Primary Frozen Benchmark ($N = 32$ Tasks): Evaluates H_1–H_4 on the frozen procurement benchmark (commit 9807340, SHA-256 78c2c88e), establishing predictive generalization (H_1), invariant safety (H_2), recovery lift (H_3), and workload token reduction (H_4). 2) Post-Freeze Robustness Study ($N = 32$ Tasks): Evaluates architecture transfer across three new synthetic schemas plus primary replication (four schemas total, SHA-256 b3d57636), providing preliminary evidence of mechanism robustness without altering the primary benchmark.

VIII. Conclusion

We introduced LASR-MCP, an adaptive schema resolution framework with dynamic in-tree recovery for enterprise tool-calling agents. By establishing an Invariant Foundation Floor (C_I^*), predicting task-conditioned constraint sets, and intercepting multi-error validator paths for native in-tree JSON Schema recomposition, LASR-MCP achieves 22.1% net workload-token reduction with 100.0% final structural validity on the primary frozen benchmark. Across four enterprise-style domains, LASR demonstrates 20.5%–25.2% token savings and complete 16/16 ambiguous-task recovery across evaluated cases, supporting the feasibility of a standards-aligned schema-resolution approach for MCP tool interfaces.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," International Conference on Learning Representations (ICLR), 2023.

[2] T. Schick, J. Dwivedi-Yu, S. Patil, G. Wenzek, P. Ludwig, M. Lewis, L. Zettlemoyer, and S. Riedel, "Toolformer: Language models can teach themselves to use tools," Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.

[3] Model Context Protocol Working Group, "Model context protocol specification (version 2026-07-28)," jul 2026. [Online]. Available: https://modelcontextprotocol.io/specification

[4] A. Wright, H. Andrews, B. Hutton, and G. Dennis, "Json schema: A media type for describing json documents (draft 2020-12)," jun 2022. [Online]. Available: https://json-schema.org/draft/2020-12/json-schema-core.html

[5] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive apis," arXiv preprint arXiv:2305.15334, 2023.

[6] P. Bryan, K. Zyp, and M. Nottingham, "Javascript object notation (json) pointer," RFC 6901, 2013.

[7] T. Gan and Q. Sun, "Mitigating prompt bloat in llm tool selection via retrieval-augmented generation," arXiv preprint arXiv:2505.03275, 2025.

[8] X. Fei, X. Zheng, and H. Feng, "Mcp-zero: Active tool discovery for autonomous llm agents," arXiv preprint arXiv:2506.01056, 2025.

[9] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian et al., "Toolllm: Facilitating large language models to master 16000+ real-world apis," arXiv preprint arXiv:2307.16789, 2023.

[10] A. Sadani and D. Kumar, "Tool attention is all you need: Dynamic tool gating and lazy schema loading for eliminating the mcp/tools tax in scalable agentic workflows," arXiv preprint arXiv:2604.21816, 2026.

[11] F. Sakizli, "Tscg: Deterministic tool-schema compilation for agentic llm deployments," arXiv preprint arXiv:2605.04107, 2026.

[12] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "Llmlingua: Compressing prompts for accelerated inference of large language models," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023, pp. 13 358–13 376.

[13] Z. Pan, Q. Wu, H. Jiang, M. Meng, Y. Yang, and L. Qiu, "Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression," in Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 8252–8269.

[14] Y.-e. Cho, "Schemarouter: Field-aware tool routing for efficient heterogeneous agentic rag," arXiv preprint arXiv:2608.21375, 2026.

[15] Z. Zhai, X. Tan, G. Zou, X. Wang, and W. Zhang, "Hyperagent: Planning and acting over tool-schema hypergraphs for tool-use llm agents," arXiv preprint arXiv:2608.02650, 2026.

[16] H. Xia, Y. Li, C. Du, M. Song, and W. Li, "Toolspec: Accelerating tool calling via schema-aware and retrieval-augmented speculative decoding," arXiv preprint arXiv:2604.13519, 2026.